Document Number:	p2921r0
Date:	2023-06-16
Target:	LEWG
Reply to:	gorn@microsoft.com

p2921r0: Exploring std::expected based API alternatives for buffer_queue

This paper explores extending interface of Buffered Queue (https://wg21.link/p0260r6) with APIs returning `std::expected` as opposed to taking `std::error_code`.

Summary of the relevant buffer_queue APIs

```
enum class conqueue_errc { success = 0, empty, full, closed };
class conqueue_error : runtime_exception { ... };
... make_error_code, make_error_condition, conqueue_category, ...


template <class T, class Alloc = std::allocator<T>>
class buffer_queue {
public:
   ...
  // modifiers
  void close() noexcept;

  T pop();
  std::optional<T> pop(std::error_code& ec);
  std::optional<T> try_pop(std::error_code& ec);

  void push(const T& x);
  bool push(const T& x, error_code& ec);
  bool try_push(const T& x, error_code& ec);

  void push(T&& x);
  bool push(T&& x, error_code& ec);
  bool try_push(T&& x, error_code& ec);
};
```

`T pop()` - throws if queue is empty and closed, blocks if the queue is empty

`void push(T)` - throws if queue is closed, blocks if the queue is full

APIs that take error_code parameter, would set `error_code` parameter to `conqueue_errc::closed)` instead of throwing an exception.

Additionally, `try_push` and `try_pop` APIs never block and would report errors `conqueue_errc::full` and `conqueue_errc::empty` respectively.

Exploring alternative APIs shapes

After discussion in LEWG in Varna 2023, we were asked to explore restyling of the APIs that take `std::error_code` parameter with `std::expected` returning ones, which we dutifully did and present the results below.

Encoding expected variant with std::nothrow parameter

To distinguish between throwing and expected returning flavors, we use `std::nothrow_t` parameter for disambiguation (alternative was to change the name to, say, `push_nothrow`, which we find less aesthetically appealing).

```
void push(const T&);
bool push(const T&, error_code& ec);
  vs
void push(const T&);
auto push(const T&, nothrow_t) -> expected<void, conqueue_errc>;
```

non-throwing: status-quo	non-throwing: expected with nothrow

`std::error_code ec;`
`if (q.push(5, ec))`
`  return;`
`println("got {}", ec);`
	`if (auto result = q.push(5, nothrow))`
	`  return;`
	`else`
	`  println("got {}", result.error());`

The benefit of expected here is that we don't have to declare ec before calling the API.

Status quo vs only expected based APIs

Another alternative suggested to us by some LEWG members was to eliminate the throwing versions altogether and replace them with the expected returning ones.

```
void push(const T&);
bool push(const T&, error_code& ec);
  vs
auto push(const T&) -> expected<void, conqueue_errc>;
```

non-throwing: status-quo	non-throwing: expected only

`std::error_code ec;`
`if (q.push(5, ec))`
`  return;`
`println("got {}", ec);`
	`if (auto result = q.push(5))`
	`  return;`
	`else`
	`  println("got {}", result.error());`

throwing: status quo	throwing: expected only

```
q.push(5);
  ...
catch(const conqueue_error& e)
```

```
q.push(5).or_else([](auto code) {
  throw conqueue_error(code);
});
  ...
catch(const conqueue_error& e)
```

throwing: status quo

```
q.push(5);
  ...
catch(const conqueue_error& e)
```

throwing: expected only (awkward exception type)

```
q.push(5).value();
...
catch(const
   bad_expected_access<conqueue_errc>& e)
```

This alternative disfavors an exception throwing and inconsistent with standard library containers. We consider this to be an inferior to the alternative offered in the previous section.

Protection against consumption of values

While in our paper we suggested to follow the precedent of `try_emplace` that guarantees that:

> If the map already contains an element whose key is equivalent to k, *this and args... are unchanged.

We recommend to follow `try_emplace` example in this regard as we do not envision that providing this guarantee for the `buffer_queue` will be a burden for the implementors.

Nevertheless, we will explore in this section alternative API shapes:

```
bool try_push(T&& x, error_code& ec); or
auto try_push(T&& x, nothrow_t) -> expected<void, conqueue_errc>;

vs

auto try_push(T&& x, nothrow_t) -> expected<void, std::pair<T, conqueue_errc>>
```

unchanged guaranteed

```
T val = get_value();
if (auto result = q.try_push(std::move(val)))
  return;
else
  println("failed {}, value {}",
     result.error(), val);
```

return back

```
if (auto result = q.try_push(get_value()))
   return;
else
  println("failed {}, value {}",
     result.error().second,
     result.error().first);
```

Returning back the value, impose a burden on the user and forces even more awkward catch clauses if bad_unexpected_access is thrown in response to `expected<T>::value()` if the expected stores error, since now error encodes the value as well.

Staying with guarantees similar to `try_emplace` remains our preferred alternative.

Linters and coding guidelines

There is another consideration brought up by LEWG is that clang-tidy warns on use after std::move. Luckily, clang-tidy recognizes try_emplace being special and have special comment in the documentation:

https://clang.llvm.org/extra/clang-tidy/checks/bugprone/use-after-move.html#move

> There is one special case: A call to std::move inside a try_emplace call is conservatively assumed not to move. This is to avoid spurious warnings, as the check has no way to reason about the bool returned by try_emplace.

For C++26, clang-tidy will need to be updated to include relevant buffer_queue APIs that do not consume the value on failure.

Coding Guidelines

Andreas Weis shared that upcoming Misra C++202x will be having a rule that bans use after move.

C++ Core Guidelines also has a rule ES.56 that

> Usually, a std::move() is used as an argument to a && parameter. And after you do that, assume the object has been moved from (see C.64) and don't read its state again until you first set it to a new value.

We may choose to update the guidelines to carve an exception to APIs like `try_emplace` rather than pessimizing `try_` API requiring incurring a move-construction as opposed to a promise not to touch the `T&&` arguments on failure.

Other APIs

Assuming that we are restyling non-throwing versions of the APIs along the lines that takes `std::nothrow` for disambiguation only, and follow the precedent of `try_emplace` with regard to not consuming `T&&` values on failure to push, the APIs will look like:

```
void push(const T&);
void push(T&&);

expected<void, conqueue_errc> push(const T&, nothrow_t);
expected<void, conqueue_errc> push(T&&, nothrow_t);

expected<void, conqueue_errc> try_push(const T&);
expected<void, conqueue_errc> try_push(T&&);

T pop();
expected<T, conqueue_errc> pop(nothrow_t);
expected<T, conqueue_errc> try_pop();
```

If the policy of LEWG would be to always add nothrow_t argument to `std::expected` returning APIs, `try_` versions will become:

```
expected<void, conqueue_errc> try_push(const T&, nothrow_t);
expected<void, conqueue_errc> try_push(T&&, nothrow_t);
expected<T, conqueue_errc> try_pop(nothrow_t);
```

Pop examples

To complete the pictures let's compare how the status quo and `nothrow_t` versions of the API fare in the pop scenarios:

Drain the queue with blocking

Status quo:

```
error_code ec;
while (auto val = q.pop(ec))
   println("got {}", *val);
```

expected based ones (one line shorter and no unneeded ec)

```
while (auto val = q.pop(nothrow))
  println("got {}", *val);
```

Drain the queue without blocking

Status quo:

```
error_code ec;
while (auto val = q.try_pop(ec))
   println("got {}", *val);
if (ec == conqueue_errc::closed)
  return;
// do something else.
```

std::expected based has unfortunate duplication since we want to know why the pop failed and we have to move val out of the loop condition.

```
auto val = q.try_pop();
while (val) {
  println("got {}", *val);
  val = q.try_pop();
}
if (val.error() == conqueue_errc::closed)
  return;
// do something else
```

The surprising observation we can make is that `std::expected` based API do not appear to be a clearcut winners and only offer marginal improvement over `std::error_code` based ones and only in some

scenarios.

Conclusion

Based on this thought experiment. We did not find `expected` based API a clear improvement over the status quo.

References

1. p1958r0: C++ Concurrent Buffer Queue
2. p0260r6: A proposal to add a concurrent queue to the standard library
3. p2882r0: An Event Model for C++ Executors
4. p2300r7: std::execution